

Anexo: Memoria del proyecto "Estadísticas Inteligentes"

1. Justificación del proyecto

En este anexo describo mi proyecto **Estadísticas Inteligentes de Restaurante**, basado en la propuesta aceptada. Mi objetivo ha sido construir una aplicación web que:

- muestre estadísticas de ventas de un restaurante,
- reciba datos estructurados en formato JSON desde una API,
- active la carga de datos mediante un botón "Cargar datos",
- muestre esos datos en forma de gráficos interactivos.

La propuesta definió la experiencia de usuario y la arquitectura que implementé:

- Frontend ligero con **HTML, CSS y JavaScript**.
- Backend serverless con **AWS Lambda, API Gateway** y una base de datos en la nube.
- Hosting estático de la interfaz en **AWS S3**.
- Visualización con librerías de gráficos como **Chart.js**.

Este planteamiento coincide con la implementación actual del proyecto, que ya incluye un frontend estático con un botón de carga y una API real en AWS.

2. Despliegue real en AWS y arquitectura serverless

No me quedo en una arquitectura teórica: la solución se ha pensado para funcionar con servicios reales en AWS.

- El frontend se puede alojar como sitio estático en un bucket de **Amazon S3**.
- La API de datos utiliza un endpoint público de **Amazon API Gateway**.
- La lógica del backend puede ejecutarse en **AWS Lambda**.
- Para un diseño típico serverless, el almacenamiento de datos recomendado sería **DynamoDB**, aunque también justifico el uso de **RDS MySQL** por requisitos académicos.

Por qué serverless

- Menor administración de servidores.
- Escalado gestionado automáticamente.
- Paga por uso real, sin instancias ociosas.
- Ideal para demos y proyectos académicos donde el volumen no justifica infraestructura compleja.

Justificación de la base de datos

- **DynamoDB** es la opción natural para serverless: acceso rápido, esquema flexible y escalado automático.

- **RDS MySQL** se puede justificar si existe un requisito académico que obligue a usar SQL relacional. Yo argumento que esta elección responde a ese requisito académico, y no porque sea la opción ideal para una arquitectura serverless pura.
-

3. Requisitos funcionales (RF) y no funcionales (RNF)

Requisitos funcionales claros y medibles

- **RF1:** Al pulsar el botón "**Cargar datos**", la aplicación debe solicitar datos al endpoint y dibujar los gráficos.
- **RF2:** La aplicación debe gestionar el estado de carga, mostrando un indicador mientras se obtiene la respuesta.
- **RF3:** Debe mostrar un mensaje de error si la API responde con fallo, si la red no está disponible o si el JSON es inválido.
- **RF4:** Debe mostrar al menos cuatro gráficos distintos: ingresos por categoría, pedidos por hora, artículos más vendidos e ingresos diarios.

Requisitos no funcionales claros y medibles

- **RNF1:** Tiempo de respuesta del endpoint `/sales` inferior a 500 ms en un escenario demo sencillo.
 - **RNF2:** Disponibilidad del frontend de al menos 99.5% siempre que el bucket S3 y el endpoint sean accesibles.
 - **RNF3:** Coste mensual estimado bajo, menor a 20-30 € para un uso de prueba/demostración.
 - **RNF4:** Documentación de integración frontend-backend disponible para que otro desarrollador entienda el flujo.
-

4. Modelo de datos: básico pero razonado

El modelo actual se centra en dos entidades principales:

- **products**
- **sales**

Por qué es básico

Esta simplificación es intencional. Para una práctica de entrega académica es preferible:

- mantener el alcance gestionable,
- centrar la solución en el flujo de datos y visualización,
- evitar complejidad excesiva que dificulte probar la demo.

Con estas dos entidades ya se pueden extraer métricas relevantes y construir un dashboard significativo.

Cómo se escala el modelo

El modelo es escalable y se puede ampliar con entidades adicionales como:

- `restaurants`
- `employees`
- `tables`
- `tickets`
- `payment_methods`
- `categories`
- `customers`

En un escenario multi-restaurante, la forma correcta es añadir un campo `restaurant_id` a `products` y `sales`, o usar alguna estrategia de multi-tenancy.

5. Integración frontend-backend

Flujo de integración real

- El usuario pulsa **"Cargar datos"**.
- `main.js` hace una petición `fetch()` a `https://9nccdykio2.execute-api.eu-north-1.amazonaws.com/prod/sales`.
- Se valida el código HTTP y se maneja el timeout.
- Se parsea el JSON, se extraen los registros y se pasan a `Chart.js`.
- Los gráficos se actualizan dinámicamente en la página.

Documentación de la API

El endpoint devuelve un JSON con un arreglo `salesRecords`. Cada objeto puede incluir:

- `date`
- `totalRevenue`
- `revenueByCategory`
- `ordersByHour`
- `itemsSold`

Este formato permite renderizar los diferentes gráficos del dashboard.

6. Costes

Estimación de costes para una demo

- **S3**: muy bajo, casi gratuito para unos pocos MB de estático.
- **API Gateway / Lambda**: pago por invocación y tiempo de ejecución; en un uso ligero es económico.
- **DynamoDB**: barato en baja demanda si se usa modo on-demand.
- **RDS MySQL**: más caro que DynamoDB; se justifica solo si es un requisito académico.

Optimización de costes

- usar S3 + CloudFront para frontend estático,
 - limitar el tamaño y tiempo de ejecución de Lambda,
 - usar DynamoDB on-demand o autoscaling en lugar de pago fijo,
 - si se usa RDS, elegir instancias pequeñas y gestionar snapshots.
-

7. Seguridad y exposición de información

Endpoint `/sales` sin autenticación

He dejado el endpoint sin autenticación por tratarse de una demo académica. Este enfoque facilita la evaluación y la demostración.

Sin embargo, en producción sería un problema crítico porque:

- permitiría acceso a datos de ventas sensibles,
- abriría el sistema a abusos y scraping,
- impediría controlar quién consulta la información.

Exposición de infraestructura

Aunque la contraseña en la documentación puede aparecer oculta, he incluido un endpoint público y datos de la infraestructura.

He justificado este enfoque por tratarse de un proyecto académico, aunque en un producto real convendría:

- ocultar nombres de base de datos y endpoints internos,
 - usar **AWS Secrets Manager** o **Parameter Store** para secretos,
 - restringir acceso por VPC/IP y roles mínimos,
 - aplicar políticas IAM estrictas.
-

8. Definición de "inteligente"

El proyecto se llama **Estadísticas Inteligentes** aunque no incluye predicciones ni recomendaciones.

En este contexto, "inteligente" se refiere a:

- visualizaciones dinámicas que transforman datos JSON en insights,
- una interfaz que permite explorar métricas sin procesar los datos manualmente,
- un dashboard capaz de presentar comparativas y patrones de ventas.

Es un dashboard **descriptivo**, no un sistema predictivo. Yo explico que la inteligencia se entiende en términos de análisis visual y facilidad de uso.

9. Alcance y limitaciones: por qué no hay panel de administración

El dashboard carga los datos desde la API, pero no tiene un panel para:

- insertar ventas manualmente,
- importar CSV,
- conectar un TPV,
- administrar productos.

Decidí priorizar:

- la visualización de métricas,
- el flujo de datos entre frontend y backend,
- la validación de la arquitectura serverless.

En esta fase prioricé el análisis y la presentación de los datos, y la entrada de datos queda para una ampliación posterior.

10. DAFO ampliado

Fortalezas

- Despliegue real en AWS con servicios gestionados.
- Arquitectura serverless clara y bien razonada.
- Interfaz intuitiva con un botón central de carga.
- Consumo real de una API REST y transformación de JSON a gráficos.
- Documentación del flujo y del endpoint.
- Proyecto demostrable y ejecutable en un entorno real.

Debilidades

- El alcance elegido se centra en la visualización de datos y no incluye aún predicciones, recomendaciones ni comparaciones avanzadas.
- En esta versión no se ha incorporado una interfaz completa para registrar ventas, importar CSV o gestionar productos; el foco ha sido validar el modelo y la visualización.
- El endpoint `/sales` funciona sin autenticación en el entorno de demostración, lo que facilita el acceso para pruebas.
- La presentación incluye información de infraestructura necesaria para justificar el diseño, aunque en un entorno productivo esa visibilidad se gestionaría con más detalle.
- El modelo de datos es reducido, pero ofrece una base clara sobre la que se puede ampliar el sistema.

Oportunidades

- Ampliar el proyecto con predicciones de demanda y recomendaciones de menú.
- Añadir importación CSV y conectores TPV para hacer la solución más completa.
- Integrar con herramientas reales como Google Looker Studio o soluciones SaaS de hostelería.
- Aprovechar la arquitectura serverless para escalar hacia múltiples restaurantes.
- Usar ML en una fase avanzada para detectar tendencias y sugerir acciones.

Amenazas

- La competencia de soluciones TPV y dashboards SaaS ya consolidadas.
 - La necesidad de seguridad real en datos de ventas si se lleva al entorno comercial.
 - El coste asociado a infraestructuras mayores si los datos crecen.
 - El riesgo de que el proyecto se perciba como demasiado básico si no se amplía en funcionalidades operativas.
-

11. Respuestas a preguntas clave

¿Por qué el modelo solo tiene **products** y **sales**?

Lo diseñé como una demo de dashboard. Con estas dos entidades he podido demostrar el flujo completo desde la obtención de datos hasta la visualización de métricas. Reducir el modelo ayudó a mantener el prototipo funcional y claro.

¿Cómo adaptarías el modelo a varios restaurantes?

Añadiría una entidad **restaurants** y la referenciaría desde **products** y **sales** mediante un campo **restaurant_id**.

Opciones:

- multi-tenancy lógica: todas las tablas comparten la misma base de datos con **restaurant_id**.
- multi-tenancy física: bases de datos separadas por restaurante.

También incluiría control de acceso para que cada restaurante solo vea sus datos.

¿Qué ocurre si se borra un producto que ya tiene ventas asociadas?

En un esquema relacional, lo recomendable es no eliminar físicamente el producto si tiene ventas históricas. Una práctica adecuada es usar un borrado lógico (**is_active**, **deleted_at**) o aplicar **ON DELETE RESTRICT** en la relación.

Si se elimina el producto físicamente, se corre el riesgo de perder integridad histórica y de dejar ventas incompletas. Otra alternativa es desnormalizar información clave del producto dentro de cada registro de **sales** para preservar el historial.

12. Conclusión

En este anexo resumo la justificación del proyecto, su alineación con la propuesta, sus limitaciones y sus fortalezas. El trabajo es una demo funcional centrada en visualización de ventas y arquitectura AWS. Explico que el enfoque elegido responde a un alcance académico y a un primer hito en el desarrollo.